

FlowLang Reference Sheet

Syntax & Operators

```
[selectors...] <operator> [targets...] [-flags...];
```

Op	Type	Description	Length
->	Sub	Replaces match with target.	Changes
-->	Over	In-place overwrite. Supports . wildcard.	Same
>	Ins	Inserts target <i>before</i> match.	Grows
><	Del	Deletes match. (No target needed).	Shrinks

Selectors & Targets Example:

```
"AAB" -> "BAA";      # Regex string literals
ABA --> BAB;          # Char literals (ASCII/numerical)
A.B --> .C.;          # Dot (.) wildcard (-1 skip)
(65, 66) -> (66, 65); # Evaluated numerical tuples
[0, 5] >< ;           # Range [start, end] delete
fn<1> -> tgt<"A">;    # Callables (via @import)
```

Flags Reference

(Unbracketed flags evaluate to `True`)

(A flag is really just an alias for a property with a longer name. Hence, the property name can optionally be directly used as a flag as well.)

Group & Lifecycle

Flag	Name	Description
-d	disabled	Disables the rule.
-g[id]	group	Assigns rule to group ID(s).
-gb[b]	group_break	(Def: <code>True</code>) Stops group eval if applied.
-a[b]	always_apply	Ignores group break, forces eval.
-life[n]	lifespan	Max successful applications before death.

Match Selection

Flag	Name	Description
-sr[s,e]	space_range	Spaces to check. (Def: [0,0])
-mr[s,e]	match_range	Matches per space. (Def: [0,0])
-cmp[m]	cmp	Conflict Mark: ignore (def), this , og , both .
-p_rule	p_rule	Prob [0-1] rule executes on match.
-p_space	p_space	Prob [0-1] space is evaluated.

Execution & Branching

Flag	Name	Description
-pl[n]	parallel_execution_limit	Max modifications before branch. (Def: 1)
-bl[n]	branch_limit	Max branches per tick. (Def: 0)
-crp[m]	crp	Conflict Resolve: ignore (def), skip , break , branch , branch_nbl .
-bo[o]	branch_origin	Branch origin: prev (def) or current .
-nct[b]	no_causality_tracking	Disables causal DAG tracking for rule.
-nib[b]	no_initial_branch	Modifies space in-place, no new gen.
-nds[b]	no_delta_submit	Hides new space from output event.

Flag Scoping Precedence

Instruction > Block > Global

```
-pl[inf] -mr[0,inf]           # 1. Global (sets defaults)
(-g[1] -gb) (                 # 2. Block (applies to all inside)
  "AAB" -> "BAA";
  "BB" >< -life[10];           # 3. Instruction (overrides)
)
```

Directives

Evaluated in 3 strict phases. **Order matters: Place `@init` before `@evolve` !**

Phase 1: Macro Expansion (Compile-Time)

- `@macro("path", *args, **kw);` Inlines file/preset AST.

Phase 2: Initializers (Pre-Compile)

- `@mem("vector"|"nd");` Sets memory backend.
- `@regex_for_literal_selectors(bool);` String matching mode.
- `@import("path.py", *names);` Loads callables.
- `@reset_imports();` Clears imported callables.

Phase 3: Program Scope (Post-Compile)

- `@init("AB" | (1,2) | "f.npy");` Initializes universe.
- `@evolve(n);` Steps simulation `n` ticks.
- `@regress(n);` Undoes `n` events.
- `@clear();` Resets to `events[0]` .
- `@merge(group_id);` Chains group rules sequentially.
- `@compress(group_id);` Disables inert overwrite rules (e.g., `A-->A`).
- `@p_seed(n);` Seeds random engine.
- `@self.prop.method();` Direct interpreter escape hatch.

Common Setups

Other flow files (e.g. `.flow` , `.pflow` , `.preset`) may be expanded in the current file using the `@macro("file_path", *args, **kwargs)` directive. Additionally, built-in presets or flows may be expanded using the path `"/lang/<name of flow or preset>"` (for cellular automata, `/lang/ca.flow` and `/lang/ca.flow` is used, for instance).